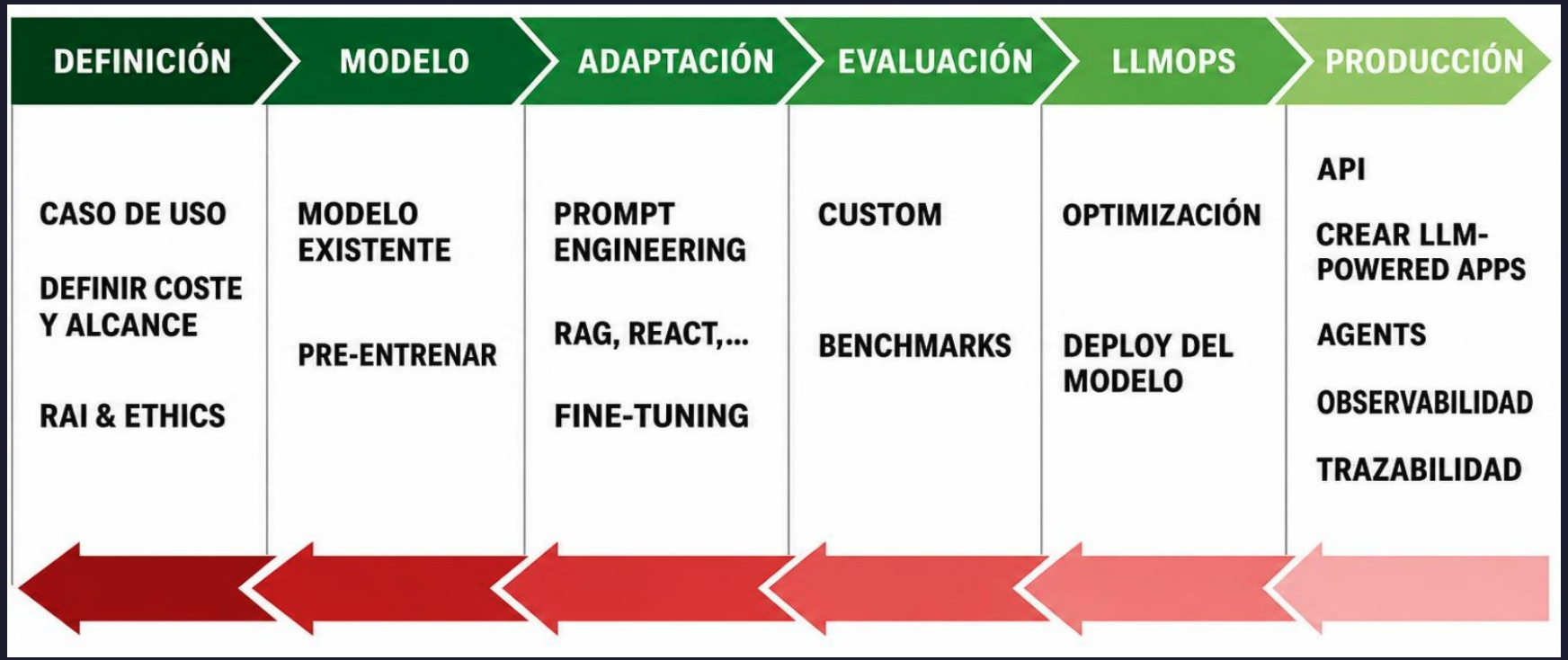
keep coding

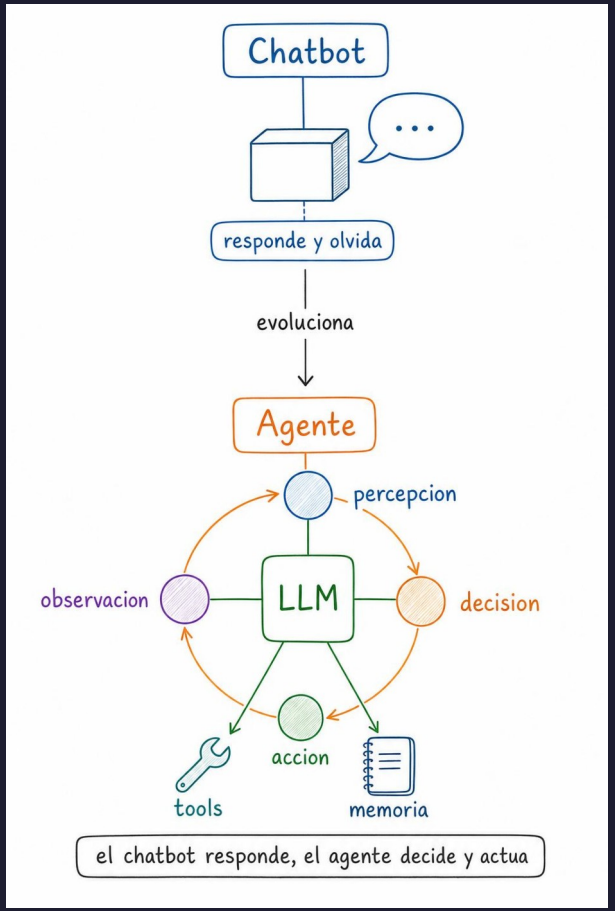
# AI ENGINEERING

## Agentes con LangChain


DEFINICIÓN
MODELO
ADAPTACIÓN
EVALUACIÓN
LLMOPS
PRODUCCIÓN
CASO DE USO
DEFINIR COSTE Y ALCANCE
RAI & ETHICS
MODELO EXISTENTE
PRE-ENTRENAR
PROMPT ENGINEERING
RAG, REACT,...
FINE-TUNING
CUSTOM
BENCHMARKS
OPTIMIZACIÓN
DEPLOY DEL MODELO
API
CREAR LLM-POWERED APPS
AGENTS
OBSERVABILIDAD
TRAZABILIDAD

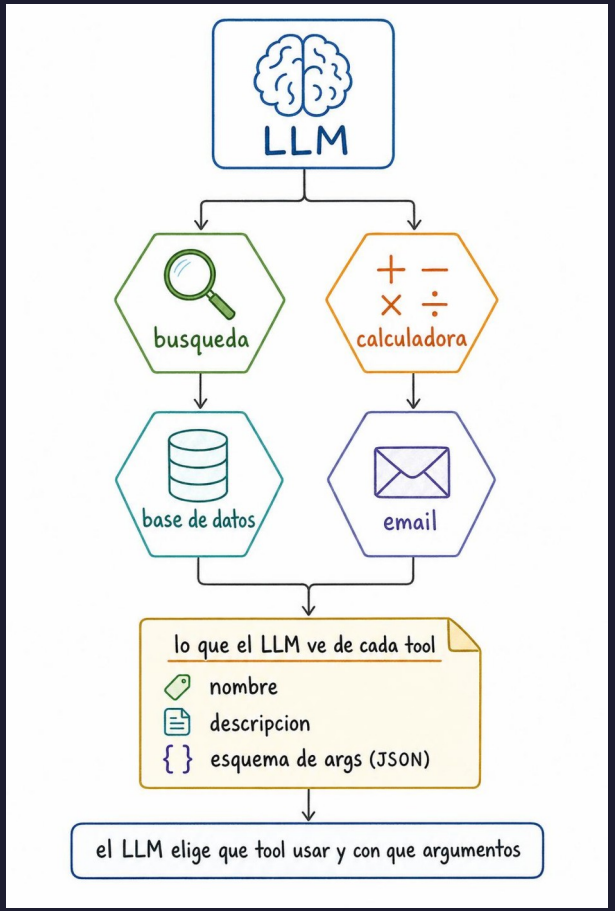
# De chatbot a agente

- Chatbot · responde texto · stateless · sin acción externa
- Agente · LLM + tools + loop · decide y observa el resultado
- El bucle · perception, decision, action, observation hasta la meta
- El LLM no ejecuta · genera la intención, el runtime la ejecuta



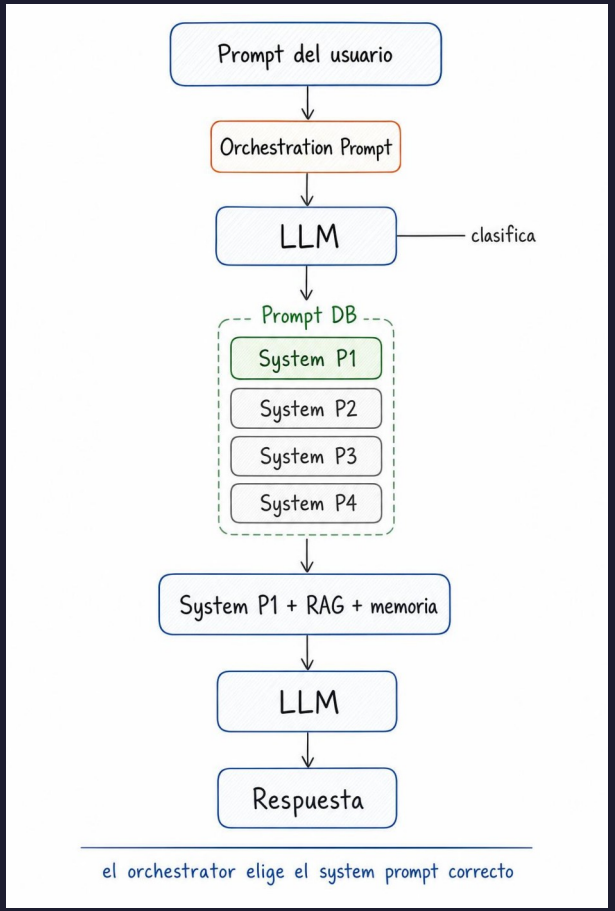
# Tools · function calling

- Tool · función Python con @tool · name · description · args_schema
- El LLM ve la firma, no el cuerpo · description + JSON schema de args
- bind_tools() · function calling nativo de OpenAI, Anthropic, Gemini
- Tool routing · el LLM elige qué tool y con qué argumentos
- Una buena description vale más que el mejor modelo



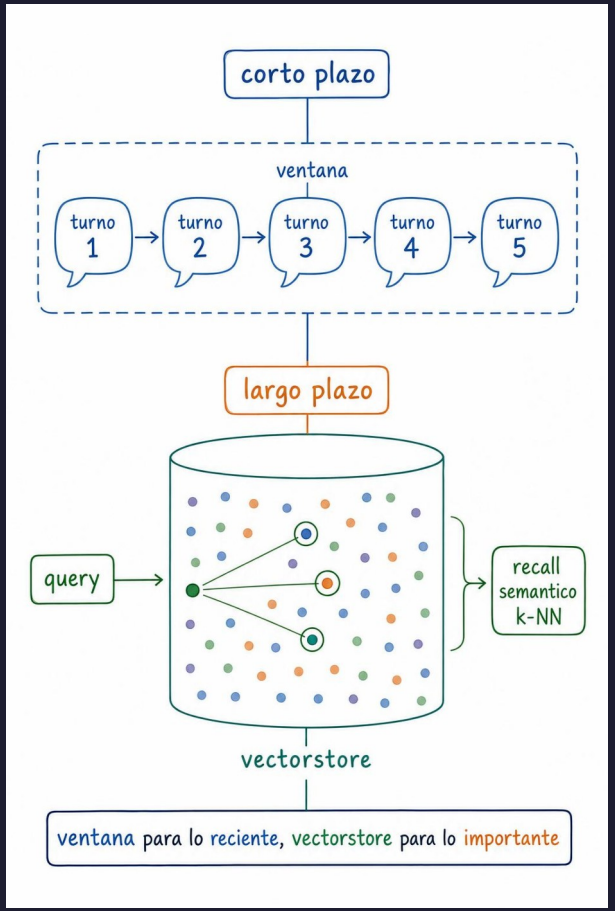
# Prompt orchestration

- Prompt DB · varios system prompts especializados, uno por caso de uso
- Orchestration prompt · un LLM clasifica la petición y elige el system prompt
- El system prompt elegido se enriquece con RAG y memoria
- Ese prompt compuesto va al LLM final, que genera la respuesta
- Un prompt enfocado por caso rinde más que un mega-prompt para todo



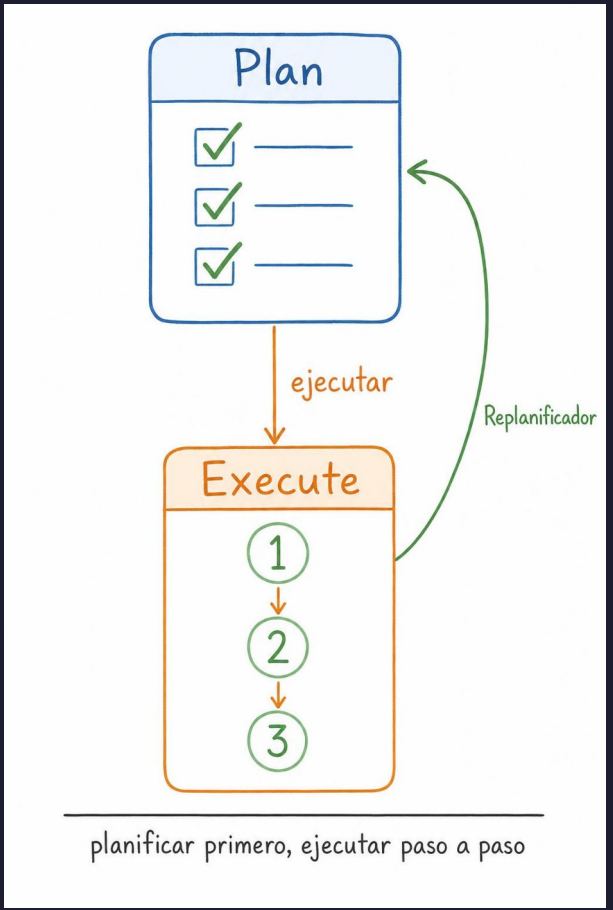
# Memoria del agente

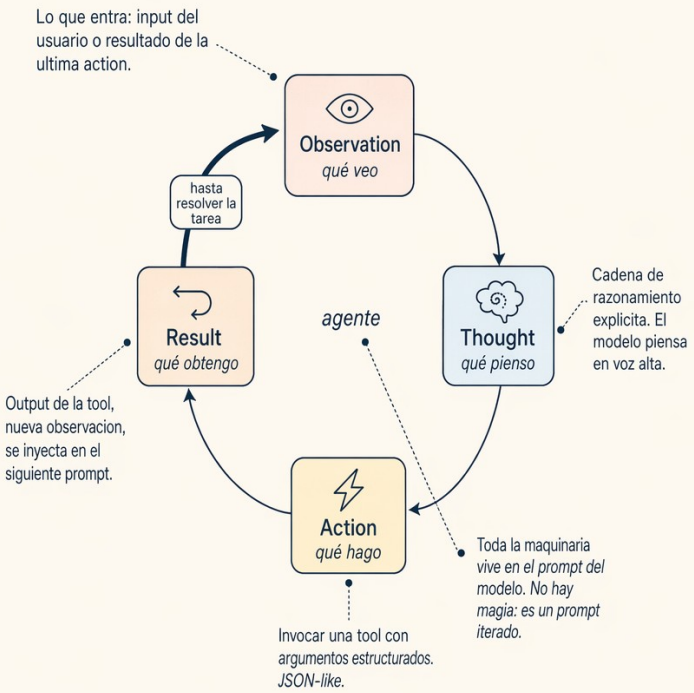
- Corto plazo · ventana de los últimos N turnos · O(N) tokens
- Resumen · un LLM condensa el historial · barato, pierde detalle
- Largo plazo · cada turno como embedding en un vectorstore
- Recall semántico · k-NN sobre la query · recupera solo lo relevante
- Sin memoria no hay agente · solo un chatbot con amnesia

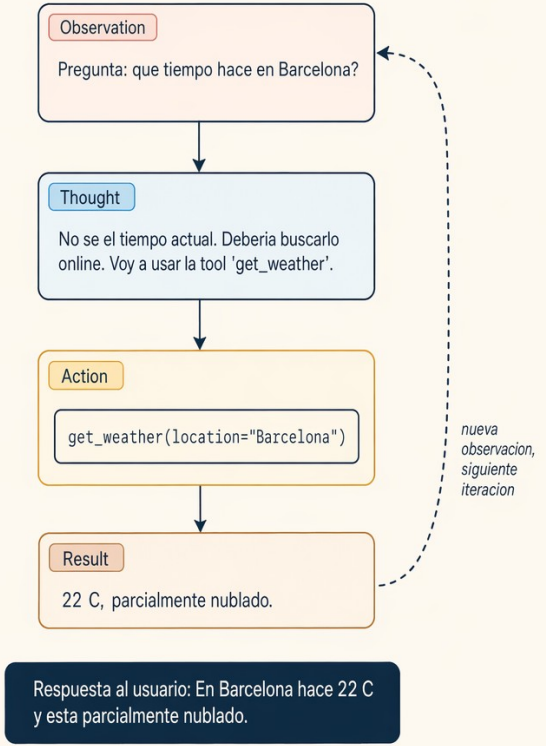


# Plan-and-Execute

- Plan-and-Execute · plan completo primero, ejecución paso a paso
- Planner · un LLM produce la lista ordenada de subtareas
- Executor · cada paso lo resuelve un sub-agente o chain
- Replanner · revisa el plan cuando un paso falla o aparece info nueva
- ReAct improvisa turno a turno · Plan-and-Execute se compromete con un plan



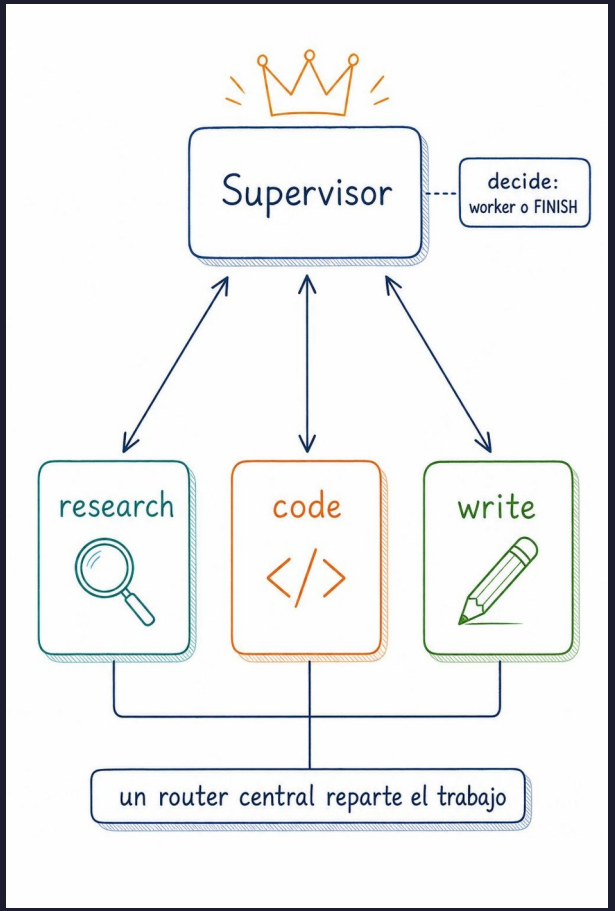



*Paper: Yao et al. 2022, ReAct: Synergizing Reasoning and Acting in LLMs*

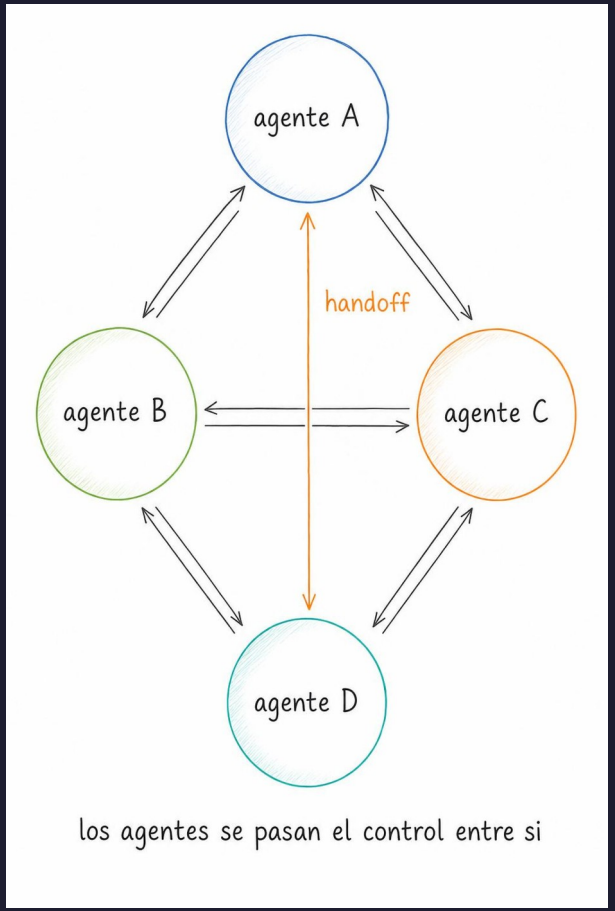
# Multiagentes · supervisor

- Topología jerárquica · un Supervisor LLM enruta a especialistas
- Cada worker · su subgrafo, sus tools, su prompt de dominio
- El supervisor recibe el output y decide: otro worker o FINISH
- langgraph-supervisor · create_supervisor(agents, model, prompt)
- Un router central · simple de razonar, fácil de depurar



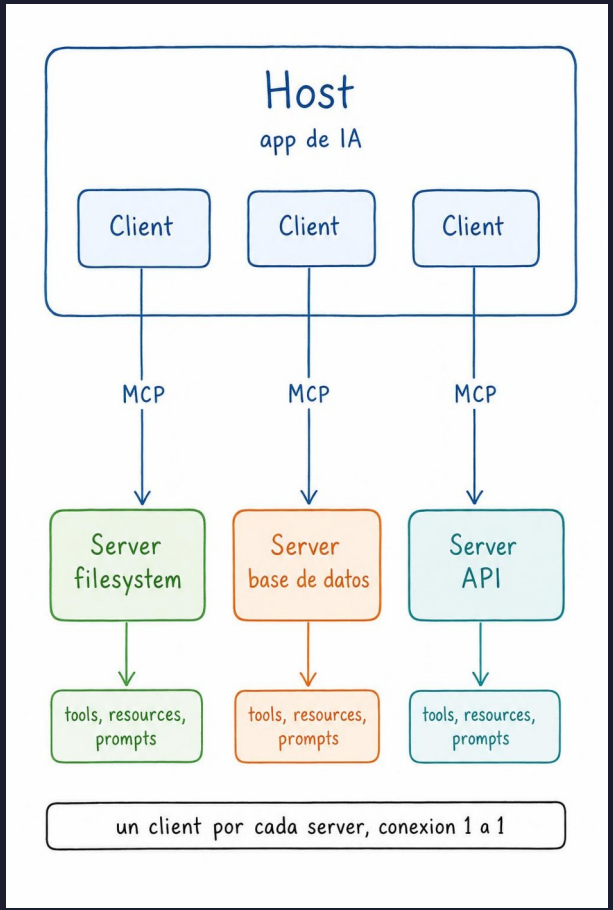
# Multiagentes · swarm

- Topología peer-to-peer · los agentes se traspasan el control
- Handoff tool · cada agente expone transfer_to_otro como tool
- Sin router central · el agente activo decide cuándo delegar
- langgraph-swarm · guarda active_agent en el state compartido
- Supervisor para control · swarm para flujos abiertos



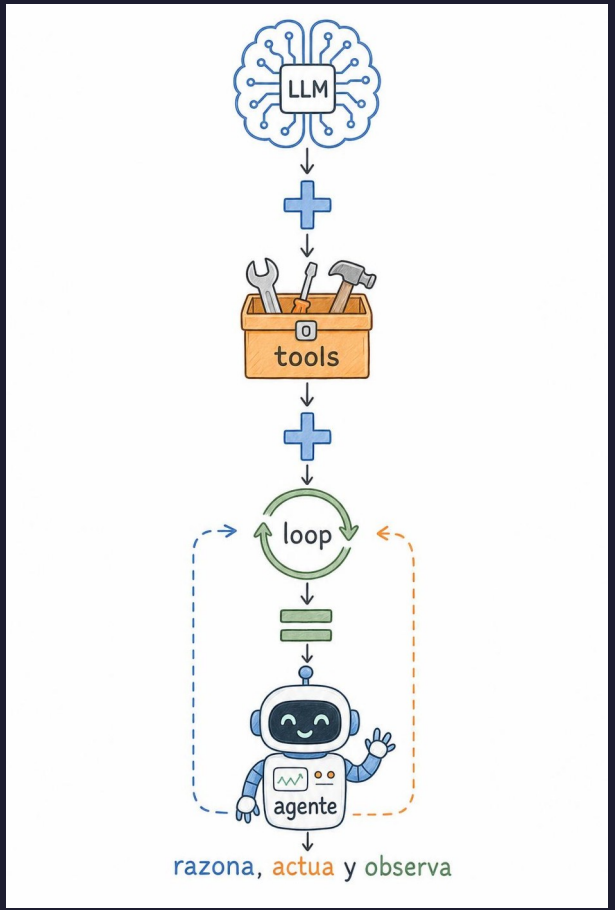
# MCP

- MCP · protocolo abierto de Anthropic (2024) · conecta el LLM con tools y datos
- Host · la app de IA · Claude Desktop, IDE, agente · arranca los clients
- Client · vive dentro del host · mantiene una conexión 1:1 con un server
- Server · expone tools, resources y prompts · stdio local o streamable HTTP remoto
- Un server vale para cualquier host · MCP es el USB de las tools



# Recap

- Agente = LLM + tools + loop · razona, actúa y observa
- ReAct para lo simple · Plan-and-Execute para tareas largas
- LangGraph cuando necesitas control, persistencia o streaming
- Multiagente solo si la tarea se descompone limpia
- MCP para conectar tools sin reescribir integraciones




keep coding